Programming Assignment #5: Boggle - Testing Report

Dinesh Nalliah and Jessica Xu

November 3, 2026

General

Writing our own tests taught us as much as receiving reviews did. To make the tests reliable, we had to design small controlled boards where we knew exactly which words should be found, and that made us think carefully about edge cases. These included backtracking out of a dead end, reusing a cell, wraparound at the edges, a 1x1 board, and a null search tactic. It also made us read the interface more closely than we had while implementing. For example, getLastAddedWord() must return null before any word is added, points are given as (row, column), and getAllWords() must return every valid word on the board.

Testing other submissions showed us how easily two correct-looking implementations can disagree on a convention. One submission found and scored every word correctly but returned its coordinates transposed, which caused four of our tests to fail. That made us double-check our own Point order against the spec. We also had to separate genuine bugs from convention mismatches, and to say plainly when a failing test had stopped early and skipped its later assertions.

The reviews we received showed that passing your own tests is not the same as being correct. Our tests used lowercase boards and always loaded the dictionary first, so they never exercised the cases one reviewer used to find real bugs.

Peer Review Ranking

1. Report 0 (most useful). This review found three real bugs and gave exact steps to reproduce each one. On the uppercase sample board, addWord("lean", 0) returned 0 and getAllWords() returned nothing, but the same board in lowercase found all 42 words. After “lean” was added, getAllWords() stopped including it. Calling contains, isPrefix, or iterator() on an unloaded dictionary throws a NullPointerException. Comparing the uppercase and lowercase boards was a smart way to isolate the first bug, and the reviewers prioritized the fixes by impact. The goals and results were clear. The main weakness is that the methodology section is brief as it didn't specify what tests were run. It still led directly to every fix we made.

2. Report 1. This was the clearest and best-organized review. It explained its custom dictionary and board, why those inputs were chosen, and which behaviors each test covered, and it added targeted tests for backtracking and cell reuse after reviewing its own suite. The methodology was thorough within its scope, and its suggestions for future tests were reasonable. However, all 15 tests passed, so it didn’t help us find any bugs. It gave us confidence in our core search logic, but not much to act on.

Revisions Made

We fixed all three issues from Report 0:

- Case handling: setGame now converts every board letter to lowercase, and addWord lowercases the guess (and rejects null) before checking it. Uppercase boards and guesses now behave the same as lowercase ones.

- getAllWords(): both search tactics now add every valid word to the returned collection, including words players have already found. Player-found words only affect whether the computer is credited, through the foundByPlayer check, rather than whether the word is returned.

- Unloaded dictionary: GameDictionary now tracks an isLoaded flag. Before a file is loaded, contains and isPrefix return false, and iterator() returns an empty iterator instead of throwing.

We also checked the two areas Report 0 suggested for future testing. addWord only updates the saved path on success, so an invalid guess leaves the last path intact, and our test suite confirms this. newGame and setGame both reset player words, scores, and the saved path.